

Validation & réglage *sans tricher*

Comparer des modèles et régler les hyperparamètres honnêtement · Marianne A. · compagnon de M1 / M4B1

Le problème. Dès qu'on *compare* des modèles (Ridge vs RandomForest vs HistGradientBoosting en M4B1) ou qu'on *règle* des hyperparamètres, on prend des décisions à partir des données. Si ces décisions s'appuient sur le **test**, le score final devient mensonger. La parade : un **jeu de validation** (ou de la validation croisée) sur le train, et un **test scellé** qu'on ne touche qu'une fois, à la toute fin.

Règle Le test ne sert **jamais** à choisir quoi que ce soit — ni le modèle, ni les hyperparamètres, ni le seuil. Il ne fait que *constater* la performance du choix final.

1 Les trois jeux : à quoi sert chacun

TRAIN (~60 %)

apprend les paramètres du modèle (.fit)

on peut le revoir autant qu'on veut

VALIDATION (~20 %)

choisit le modèle & les hyperparamètres

sert à comparer / arbitrer

TEST (~20 %)

— SCELLÉ —

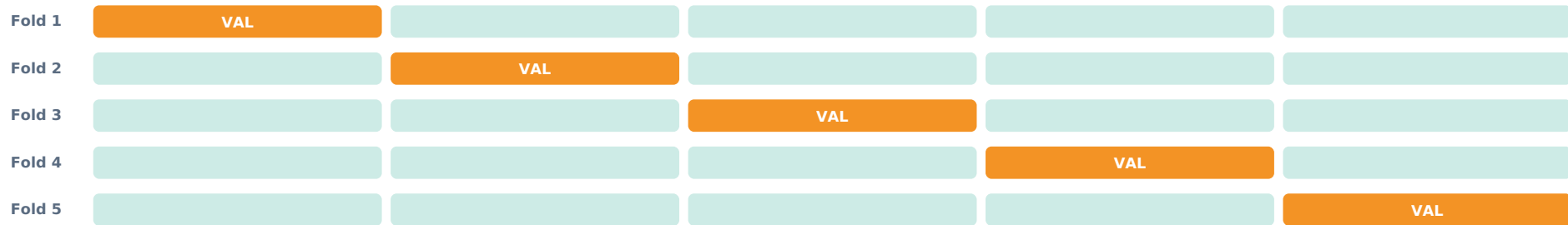
verdict final, 1 fois

jamais avant la fin

Pourquoi trois et pas deux ? Si on règle les hyperparamètres directement sur le test, on finit par « apprendre » le test à force d'essais — le score ne reflète plus la vraie généralisation. Le jeu de validation absorbe ces choix ; le test reste vierge pour une mesure honnête.

2 La validation croisée (k-fold) : quand un seul split ne suffit pas

Avec **peu de données**, un unique jeu de validation est instable (le score dépend du tirage). La **validation croisée** découpe le train en k parts : on entraîne sur $k-1$ parts et on valide sur la dernière, en tournant, puis on **moyenne** les k scores. Résultat : une estimation plus fiable et une idée de la **variance**. En classification (surtout déséquilibrée), on utilise **StratifiedKFold** pour conserver les proportions de classes.



■ entraînement ■ validation → score final = moyenne des 5 scores ($\pm$ écart-type)

3 Régler les hyperparamètres, proprement

On teste plusieurs combinaisons d'hyperparamètres **en validation croisée sur le train**, on garde la meilleure, puis *seulement à la fin* on mesure sur le test. Point crucial : le **préprocessing doit être dans le pipeline** pour être ré-apprié à chaque fold — sinon fuite de données.

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

grid = {"clf__max_depth": [3, 5, None], "clf__n_estimators": [200, 500]}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

search = GridSearchCV(pipe, grid, scoring="f1_macro", cv=cv, n_jobs=-1)
search.fit(X_train, y_train) # tout se joue sur le TRAIN
print(search.best_params_, search.best_score_)

# une seule fois, à la fin :
final = search.best_estimator_
f1_score(y_test, final.predict(X_test), average="macro") # verdict
```

OUTIL	QUAND	À SAVOIR
GridSearchCV	Peu d'hyperparamètres, grille raisonnable.	Exhaustif mais coûteux : le nb de combinaisons explose vite.
RandomizedSearchCV	Beaucoup d'hyperparamètres / grands espaces.	Échantillonne au hasard ; souvent aussi bon pour bien moins cher.
cross_val_score	Juste estimer un modèle (sans régler).	Renvoie les k scores : regarder moyenne <i>et</i> écart-type.

4 Pièges de « triche » involontaire

LE PIÈGE	SYMPTÔME	LE CORRECTIF
Préprocessing hors CV	Score de validation trop beau : le scaler/OHE a « vu » les folds de validation.	Mettre tout le préprocessing dans le Pipeline passé à la CV.
Régler sur le test	On itère jusqu'à un bon score de test → il ne veut plus rien dire.	Régler en validation ; toucher le test une seule fois, à la fin.
Oublier stratify / StratifiedKFold	Folds aux proportions de classes différentes ; scores instables.	StratifiedKFold (classif), surtout en déséquilibre.
Fuite temporelle	Sur des séries, valider sur du passé avec des infos du futur.	TimeSeriesSplit : on valide toujours sur l'après.
Sur-optimiser la validation	Des centaines d'essais → on surapprend le jeu de validation lui-même.	Limiter les essais, garder un test vraiment à part comme juge final.
Comparer sur des splits différents	« Mon modèle bat le tien » avec des découpages non identiques.	Même CV / même seed pour tous les modèles comparés.

La règle d'or. Sépare le test en premier et scelle-le · règle et compare **tout** par validation croisée sur le train · mets le préprocessing dans le pipeline pour qu'il soit ré-appris à chaque fold · choisis un *scoring* adapté (F1 macro en déséquilibre) · le test = **une** mesure finale, jamais un terrain d'essai.

À retenir. train = apprendre · validation = choisir · test = constater (une fois). La validation croisée fiabilise l'estimation quand les données sont peu nombreuses. Toute décision prise en regardant le test est une fuite déguisée.

Fiche 6/13 · Série ML